

Lab 1 – Requirements Engineering and UML Use-Case Modelling

System: Automated Rubric Assignment Evaluator

Name: Samyak Sanklecha

SRN: PES2UG24CS436

Subject: Software Engineering Lab

1. Requirements Table

Domain: Campus and Academic Operations

Actors: Student, Faculty Evaluator, Peer Reviewer

Req ID	Type	Description	Priority	Acceptance Criteria	Rationale	Comments
FR-001	Functional	The system shall automatically queue uploaded student project zip files, execute pre-configured unit test suites, and generate an itemised rubric score breakdown.	High	Pass: rubric breakdown with test results appears within 10 s of upload. Fail: queue stalls, scores are missing, or tally is wrong.	Core feature — everything else in the system depends on this working.	Given
FR-002	Functional	The system shall allow a student to log in with their university credentials and submit one project zip file per active assignment.	High	Pass: student signs in, picks an open assignment, uploads zip, gets a confirmation. Fail: valid credentials rejected, or the same student can submit more than once.	Login needs to be tied to submissions so we know whose work is whose.	
FR-003	Functional	The system shall check every uploaded file for correct zip format, size under 50 MB, and required internal structure before queuing it.	High	Pass: wrong format or oversized file rejected in under 2 s with a clear error; valid zip enters the queue. Fail: bad file slips through, or a good one gets blocked.	Stops junk from reaching the evaluator and wasting test cycles.	

FR-004	Functional	The system shall let faculty evaluators create, edit, and save rubric templates with custom criteria and point weightages before an evaluation run starts.	Medium	Pass: faculty adds/edits/deletes criteria, saves, and evaluation uses the right rubric. Fail: edits don't persist, or the wrong rubric gets applied.	Not every assignment is graded the same — faculty need to control this per assignment.	
FR-005	Functional	The system shall automatically assign each submitted project to a peer reviewer and notify them with review instructions.	Medium	Pass: within 5 min of deadline, every project has a peer reviewer assigned and notified. Fail: any submission ends up with no reviewer.	Doing this by hand for a whole class is a pain — automation is the obvious fix.	
NFR-001	Non-Functional	The system shall handle up to 100 concurrent project submissions without dropping queue items or exceeding a 1.5 GB memory footprint.	High	Pass: stress test with 100 simultaneous uploads shows no lost submissions and memory stays under 1.5 GB. Fail: submissions disappear or memory goes over the limit.	Everyone submits right before the deadline — the system has to handle that without dropping anything.	Given
NFR-002	Non-Functional	The system shall encrypt all submission data and rubric scores at rest and restrict access to only the submitting student and their assigned faculty or peer reviewer.	High	Pass: accessing another student's submission returns 403; stored files are AES-256 encrypted. Fail: unauthorised access is possible.	Student work and grades are private — basic data protection is non-negotiable.	

2. UML Use-Case Diagram

System: Automated Rubric Assignment Generator

Actors: Student (primary), Peer Reviewer (sub-actor of Student), Faculty Evaluator

«include»: Submit Project → Execute Test Suite; Submit Project → Generate Rubric Score

«extend»: Notify Late Submission → Submit Project (fires only when deadline is missed)

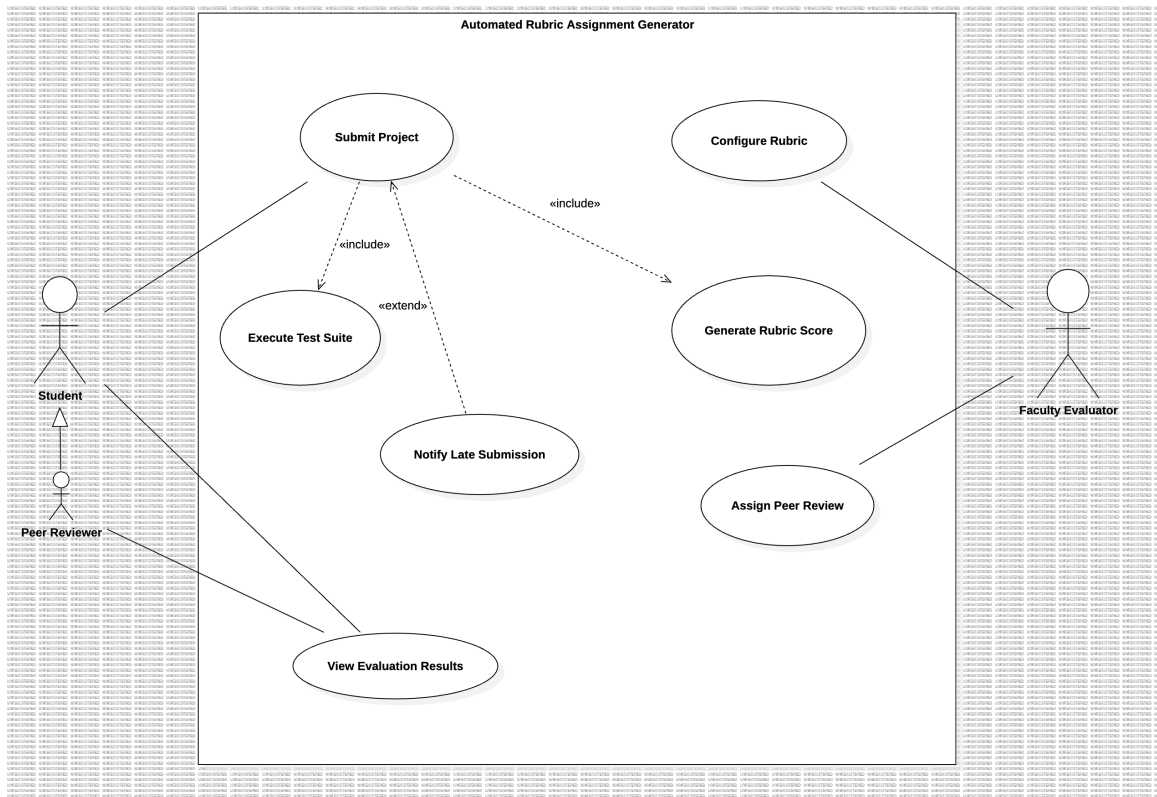


Figure 1: UML Use-Case Diagram — Automated Rubric Assignment Evaluator

3. Use-Case Flow — UC-01: Submit Project

Use Case ID: UC-01

Use Case Name: Submit Project

Primary Actor: Student

Secondary Actor: Evaluation System

Preconditions

- Student is logged into the portal.
- An assignment is currently open for submissions.
- Faculty Evaluator has already set up the rubric for this assignment.

Postconditions

- Project is queued and fully evaluated.
- Student can see an itemised rubric score breakdown on the portal.

Main Success Scenario

1. Student logs in and goes to the list of open assignments.
2. Student clicks the assignment they want to submit for.
3. Student picks their zip file and hits upload.
4. System checks the file — right format? Under 50 MB? Correct internal structure?
5. All checks pass; system drops the file into the evaluation queue.
6. System runs the unit tests pre-configured for this assignment.
7. System scores the submission against the rubric and generates a breakdown.
8. Student sees their results on the portal. Use case ends.

Alternate Flow A — Bad File (at Step 4)

- 4a1. File is not a zip or is over 50 MB.
- 4a2. System rejects it straight away and shows a specific error (e.g. "File must be a .zip under 50 MB").
- 4a3. Student fixes the issue and re-uploads.
- 4a4. If they fail 3 times in a row, uploads are blocked for that session and the faculty evaluator gets notified.

Alternate Flow B — Late Upload (at Step 3)

- 3a1. Student uploads after the assignment deadline.
- 3a2. System accepts the file but marks it as late and logs the exact timestamp.
- 3a3. Faculty evaluator is notified that a late submission came in.
- 3a4. Evaluation runs as normal — any late penalty is applied through the rubric settings.